

工业软件课程项目

需求规格说明书

项目名称：智驭能效——大型钢铁厂自适应节能系统

小组：2 课堂第 25 组

成员姓名：段冠屹、万洋、朱靖轩、王鑫瑶、
徐一、徐亿立、彭锐豪、李文卓

2026 年 6 月 8 日

1 项目概述

随着工业生产过程逐步迈向数字化和精细化管理，能源消耗数据的采集、分析与优化需求日益增长。然而，多数企业仍依赖人工管理或固定阈值告警，难以实现对能耗趋势的提前判断与跨能源类型的统一管理。同时，深度学习等时间序列预测技术迅速发展，使基于历史能耗数据的短期预测成为可能，由此推动能源管理从“被动响应”向“主动预判”转型。在这一背景下，构建一套轻量化、可扩展、可演示的能耗智能分析与预测系统，具有一定实践价值和应用意义。

本项目旨在设计并实现一个**面向中小规模工业场景的能耗优化原型系统**，通过统一的数据处理流程、可视化展示界面与深度学习预测模型，为用户提供基础能源监测和智能辅助决策功能。项目的主要目标包括：**实现能耗数据的集中处理和可视化呈现**；构建基于 LSTM 的**短期预测模型**，支持对未来能耗趋势的提前判断；基于预测结果**生成智能告警**，提高异常检测的及时性；以及交付一个**可展示的集成系统**，用于验证预测驱动的能耗管理模式的可行性。

项目内容围绕**数据处理、模型构建、前端展示和系统集成**展开。首先从公开数据源获取能耗数据，并进行清洗、字段对齐与特征构建等预处理工作；随后基于深度学习方法实现能耗预测模块，并通过简洁的 Web 界面展示能耗历史、统计指标和预测结果；同时实现基于规则与预测误差的**智能告警功能**；最后通过容器化技术完成系统集成与部署，使各模块能够在统一环境中稳定运行。项目的最终成果将是一个可运行的能耗优化原型系统，其功能覆盖数据导入、可视化、预测与告警等关键环节，能够帮助用户更直观地理解能耗变化趋势，并为进一步的智能化优化提供基础。

项目采用**Scrum敏捷开发**方式实施，通过规划、迭代开发和集成验收三个阶段逐步推进。整体周期为 6 天，分为三个迭代，每个迭代交付可运行的功能增量，从而确保项目过程透明、任务拆分明确，并能够根据开发情况及时调整规划。

2 项目预启动

2.1 项目动机与必要性

钢铁厂高炉系统是钢铁生产中能耗最高的环节，其能源消耗主要来自鼓风机电力、冷却水循环以及煤气消耗。传统管理依赖人工经验，响应滞后且缺乏系统化优化手段，导致能耗高、节能效率低。

本项目旨在开发一套高炉能耗智能监控与优化系统，通过数据采集、能耗指标计算、能耗趋势预测和优化控制建议，实现高耗能设备的动态节能与碳排放管理，为企业提供可量化的节能潜力评估与决策支持。

技术价值方面：系统集成能耗预测、优化控制与智能告警，实现高炉能耗管

理的闭环优化；**经济价值方面**：优化高炉鼓风机与冷却系统的运行参数，可降低电力和煤气消耗，实现节能成本效益；**战略价值方面**：符合国家“双碳”政策与绿色制造要求，为企业绿色生产决策提供数据支持。

2.2 项目特色与差异性

2.2.1 差异化特色

(1) 预测驱动和优化闭环

传统能源管理系统多依赖历史数据统计或固定阈值告警，缺乏前瞻性。本项目引入基于长短期记忆网络（LSTM）的能耗趋势预测模型，通过对时序能耗数据的动态学习，提前识别异常趋势。

预测结果将驱动自适应优化控制策略，实现“预测—评估—优化—验证”的动态闭环流程，提升系统的智能响应能力和能效利用率。

(2) 智能告警与自然语言报告生成（NLG）

针对工业能耗数据复杂、决策人员非技术化等痛点，本项目引入自然语言生成技术（NLG），将预测与优化结果自动转换为结构化告警信息与易于理解的优化建议。

例如，当系统识别到某一生产线的气体利用率偏低时，可自动生成类似于：“预计未来两小时热气回收效率下降 12%，建议调整炉温控制参数” 的报告，提高信息传达效率与可操作性。

	传统能源管理系统	本项目方案
优化驱动方式	依赖历史数据统计缺乏前瞻性 	基于LSTM能耗趋势预测模型，时序数据动态学习，实现“预测-评估-优化-验证”闭环控制参数调整 
智能响应能力	系统响应被动，优化策略滞后 	预测结果驱动自适应控制，动态调整参数，提高能效利用率 
告警机制	固定阈值规则告警，缺乏灵活性 依赖图表报表，人工分析为主	结合预测与NLG生成结构化告警 提供可操作建议 
数据应用范围	单能耗类型分析（电能/热能） 	自动生成自然语言报告，向非技术人员直观传达 融合电/热/气多维能源数据，形成整体能源画像 

2.2.2 市场差异性与竞争分析

目前市场上已有的部分能源管理系统（如西门子 SIMATIC Energy Manager Pro、施耐德 EcoStruxure Resource Advisor、华为数字能源管理系统（DEMS）等）多聚焦于以下方向：

- 1. 能耗采集与可视化展示：强调能源监测与数据看板；

- 2. **设备级能效分析**：提供统计报表与能效基准对比；
- 3. **碳排放追踪**：支持碳资产数据的汇总与审计。

然而，这些系统在预测性优化与智能化决策辅助方面仍存在不足，主要体现在：

- 1. 预测功能以回归分析或传统时序模型为主，**缺乏深度学习的自适应性与精度**；
- 2. 告警系统通常基于固定阈值规则，**无法根据运行状态动态调整**；
- 3. 报告生成仍依赖人工分析，**信息转化效率低**。

本项目的**差异性与创新点**主要体现为：

- 1. **融合多能耗数据源**：实现电、热、气一体化分析，形成整体能源画像；
- 2. **深度学习预测驱动优化**：以 LSTM 预测模型为核心，实现面向趋势的自适应控制；
- 3. **智能可解释告警机制**：将模型输出与自然语言生成结合，提供可读、可操作的决策支持；
- 4. **轻量化、模块化架构**：针对中小型工厂或实验场景设计，易于部署与扩展。

2.3 智能化体现

本项目的智能化主要体现在以下三个层面：

(1) 能耗预测智能化

系统基于深度学习模型 (LSTM) 对多维能耗时序数据进行建模与趋势预测。通过输入历史温度、压力、风机功率、气流量等变量，模型可输出未来 30 分钟至 1 小时的能耗趋势曲线及超标概率，为后续优化决策提供数据依据。与传统线性预测方法相比，LSTM 能够捕捉非线性动态变化及长期依赖关系，显著提升预测精度。

(2) 自适应优化与决策智能化

在预测结果的基础上，系统引入自适应优化算法，根据能耗状态、生产约束与安全阈值自动生成节能控制建议。例如，在不影响生产质量的前提下，系统可提出“降低鼓风机功率 3%”或“提高冷却水流量 2%”等可执行方案，形成预测驱动的闭环优化控制。

(3) 智能告警与 NLG 报告生成

通过自然语言生成技术 (NLG)，系统可自动将能耗异常与预测结果转化为结构化告警报告。例如：“高炉 2 号在 10:35 出现能耗上升 (+12%)，主要因冷却水温差增大。建议优化水泵功率参数，预计节能约 5.5%。”

该机制实现了数据到知识的智能转化，提升了系统的可解释性与实用价值。

2.4 初步需求分析

结合项目目标与应用场景，系统的初步功能需求如下：

分类	功能需求	说明
数据处理	数据清洗、异常值过滤、特征提取	边缘计算节点或数据预处理模块实现

预测分析	基于LSTM模型预测未来能耗趋势	输出未来能耗曲线及超标风险区间
优化建议	自适应算法生成控制参数调整方案	实现节能控制策略推荐
智能告警	自动检测能耗异常并生成NLG报告	包括原因分析与优化建议
可视化界面	实时仪表盘、能耗曲线、碳排量统计	Web端交互式可视化界面
数据存储	时序数据库	支持高频数据写入与历史查询
模型再训练	周期性模型更新与性能验证	支持持续优化与自学习

2.5 数据集来源与可获得性

本项目为课堂研究型开发，主要使用公开工业能耗与工况数据集，具体如下：

数据集名称	来源	内容说明	适用模块
UCI Industrial Energy Consumption Dataset	UCI Machine Learning Repository	工业设备能耗、功率与温度时序数据	预测模型训练
SECOM Sensor Dataset	UCI	设备运行与传感器异常监测数据	异常检测与告警
Steel Industry Energy Consumption Dataset (Kaggle)	Kaggle	钢铁行业多维能耗、炉温、风压、电流数据	主数据源
Gas Turbine CO & NOx Emission Dataset	UCI	模拟高炉热平衡与冷却系统能耗数据	模型泛化验证

数据均可通过开放数据平台获得，确保可重复性与扩展性。

2.6 预算评估与财务分析

本项目定位为教学课堂项目，经济效益主要体现在潜在的工业节能价值与计数可移植性，主要的成本类别以及费用说明如下：

成本类别	估算费用（元）	说明
硬件设备	100	用于本地服务器以及数据采集
软件与工具	0	包括数据库、可视化，模型与开发环境
人力投入	0	由项目团队人员分工完成
杂项支出	70	网络存储、数据下载等
合计	约 170 元	课堂研究项目，几乎无预算

2.7 项目进度评估

2.7.1 开发周期规划

为确保“智驭能效 (SmartEnergyMaster)”项目能够在限定周期内高效交付，项目采用**Scrum敏捷开发流程**，共设置 3 个迭代(Sprint)，每个迭代周期 2 天，总工期 6 天。进度评估以 迭代节奏控制、关键路径规划与风险缓冲 为核心，体现项目的敏捷性与执行可行性。

阶段	时间	目标	核心交付物
迭代 1	1-2 天	完成需求细化、系统架构设计与数据库环境搭建	完成需求与架构定稿，形成初步系统蓝图
迭代 2	2-4 天	完成数据采集接口、能耗计算、预测模型、可视化与告警模块的核心开发	输出核心功能原型，实现能耗采集与预测功能
迭代 3	4-6 天	进行模块联调、测试验证、系统验收与文档整理	完成系统联调与验收，提交用户手册及总结报告

2.7.2 进度控制与监控机制

项目组将会在执行中采用：

1. **每日站会 (Daily Scrum)**：每日早上 10 分钟，更新任务进度与阻碍。
2. **迭代评审 (Sprint Review)**：每个迭代结束时展示成果，收集反馈。
3. **迭代回顾 (Sprint Retrospective)**：总结问题与改进措施，优化后续流程。
4. **任务管理工具**：使用**燃尽图**实时监控剩余工作量，并采用**CodeArts**平台进行任务拆分与进度追踪。

2.7.3 总体进度可行性评估

通过 3 个高频短迭代与多角色并行协作，本项目具备良好的进度可控性与交付保障。预计可在 6 天内完成从需求分析到系统交付的完整闭环，进度偏差控制在 $\pm 10\%$ 以内，达到课堂项目规模下的高效敏捷实践目标。

2.8 潜在风险识别与分析

在项目预启动阶段，团队对潜在风险进行了初步识别与分类。结合系统架构特点、技术路线与开发周期，将关键风险分为**技术风险**、**环境风险**、**管理风险**与**协作风险** 四大类，并从中筛选出对项目影响较大的高优先级风险，为后续迭代提供参考依据。

(1) 技术风险

技术风险主要来自数据质量、模型性能与系统功能实现等方面，包括：**数据获取不完整或字段不一致**，可能影响模型训练质量；**模型预测精度不足或调参困难**，导致预测模块不稳定；**算法复杂度或可视化性能不足**，影响系统整体响应速度；智能告警逻辑不成熟，存在误报或漏报情况。

这些风险将在迭代过程中通过数据预处理规范、模型轻量化设计、参数调优流程与接口验证等方式逐步缓解。

（2）环境风险

主要体现为：**Docker与依赖环境冲突**、不同成员开发环境差异可能导致部署失败，或模型组件在容器中运行异常。

预启动阶段已规划统一基础镜像与标准化部署脚本，以降低环境差异带来的不确定性。

（3）管理风险

管理风险集中在团队分工与工作量分布方面：**任务分配不均**易导致瓶颈环节出现延误；**进度不透明**会影响迭代计划的执行。

为此，在后续Scrum 流程中将通过每日站会、燃尽图与优先级控制机制进行持续监控与调整。

（4）协作风险

协作风险主要体现在模块接口、数据格式和集成阶段：**模块间接口定义不一致**可能导致集成失败；**跨角色协作延迟**（前端、后端、模型）会影响关键路径交付。

预启动阶段已提出接口文档统一、阶段性交互测试等预防措施，确保后续迭代的稳定推进。

总体风险应对思路

虽然本项目周期较短、团队规模有限，但整体风险仍可通过一系列原则实现有效控制：

1. 在迭代初期 冻结关键接口与数据格式；
2. 对高优风险保持 持续监控，必要时调整开发优先级；
3. 建立 简化风险记录清单，即时报告阻碍因素；

通过**小步快跑、持续交付**的方式降低单次失败的影响。

3 项目启动

项目启动是项目成功的基石。本项目的启动阶段严格遵循 Scrum 敏捷管理框架。由本小组的Scrum Master与产品负责人紧密协作，共同主持了项目启动会

议、需求讨论会议和迭代计划会议。本阶段的核心目标是统一团队愿景、明确项目价值、识别关键干系人并确定首个迭代（Sprint）的可交付范围。

3.1 制定项目章程

项目章程是项目启动阶段的核心成果和顶层治理文件，标志着项目正式启动并获得授权。它明确项目的战略依据、商业理由、目标、范围、组织架构、关键干系人和资源承诺。作为连接企业战略与项目执行的纽带，项目章程为团队提供统一目标、明确约束和成功标准，使其具备制定计划、协调资源、管理风险和执行监控的权力，并为项目生命周期内的决策与评估提供依据。

3.1.1 项目名称与愿景：

项目名称：《智驭能效：大型钢铁厂自适应节能系统》（SmartEnergyMaster）。

项目愿景：构建一个面向大型钢铁厂的“预测—控制—解释”闭环智能节能管理系统，通过数据驱动的方法提升车间设备的能效管理能力，实现节能降碳目标的可量化落地。

3.1.2 商业论证与动机

政策压力：为响应国家“双碳”战略，工业企业需形成可核查的节能路径，能效优化与碳排管理成为企业核心关注点。

核心痛点：钢铁厂等流程工业车间的能源消耗占制造总能耗的 60% 以上，但传统管理依赖人工经验决策，存在调整滞后、难以处理复杂耦合因素等问题，导致能源浪费与设备风险升高。

3.1.3 项目目标与可交付成果

本项目旨在通过数据采集、能耗趋势预测和自适应控制，实现高耗能设备的动态节能与碳排管理。

主要可交付模块包括：

1. 数据接入与处理系统（数据库）
2. 能耗预测模块（基于 LSTM + Transformer 的混合模型）
3. NLG 可解释告警模块（基于“异常—原因—建议”的自然语言报告）
4. 自适应节能控制模块（强化学习/模糊控制策略）
5. 可视化与管理控制台

量化指标:

- 1) 预测精度: 能耗预测模型的 MAPE 与 RMSE 均应满足项目定义的性能阈值, 并在主要工况中保持稳定的预测能力。
- 2) 响应速度: 节能率需达到项目设定的业务目标, 并实现车间能耗与碳排放的全过程可视化、可追溯与可验证。
- 3) 节能效果: 单设备节能率达到预期目标, 同时支持全流程碳排可视化

3.2 干系人识别

在项目启动阶段, 团队共同识别了项目的内外部干系人。PO 负责管理其期望, SM 则需确保团队(特别是内部干系人)协作顺畅, 其识别的干系人及其核心关注点如下:

表 3-1 干系人识别表

分类	干系人	核心关注点 (价值需求)	对应用户故事
直接干系人 (核心用户)	企业管理层	关注碳排与 ROI 可视化, 用于年度节能审计与战略决策。	用户故事 1
	车间操作员	关注清晰、可执行的 NLG 异常告警 (含语音), 减少巡检负担和设备损坏风险。	用户故事 2
	生产调度人员	关注能耗与产能的权衡曲线, 用于在保证产能前提下优化排产。	用户故事 3
间接干系人 (影响与支撑)	IT/运维部门	关注系统稳定性、网络安全、数据库管理与日志记录。	用户故事 4
	环保/监管部门	关注碳排放量报告、节能效果的真实性与合规性。	(N/A)
	设备供应商	关注系统兼容性、设备寿命与维护反馈。	(N/A)
内部干系人 (项目团队)	PO (段冠屹)	关注功能完整性、市场契合度、可交付成果。	(N/A)
	SM (徐一)	关注项目按计划完成、团队协作顺畅、风险控制。	(N/A)
	DevTeam (全员)	关注系统性能、用户体验、模型精度。	(N/A)

3.3 启动会议

项目启动系列会议由敏捷过程负责人联合产品负责人共同组织与推进, 旨在确保各干系角色在项目早期阶段充分表达诉求并参与决策。全体团队成员参与了

会议讨论，最终形成了若干具有实际指导意义的成果，为项目执行阶段提供了明确方向和可操作的规划依据。

3.3.1 统一敏捷协作规范：

迭代节奏：确定项目周期为 3 个迭代（Sprint），每个迭代周期为 2 天。

开发假设：假定团队每位成员 100% 投入本项目，确保任务高效推进。

工具配置：确定使用**华为云CodeArts**的项目管理与任务看板模块，并在 CodeArts Repo 中制定并落实 Git 版本管理规范，完成代码仓库的初始化与权限配置。

3.3.2 明确项目范围与迭代目标

（1）范围梳理与用户故事管理

1. 需求采集与细化

团队基于 3.2 节中识别的干系人需求，在**CodeArts Req**模块召开需求讨论会与用户故事工作坊。将业务诉求拆解为用户故事（User Story），每条都采用标准格式：*As a ... , I want ... , So that ...*

同时为每个用户故事在 CodeArts Req 中创建**验收标准**（Acceptance Criteria），保证它们可验证、可测试，并可在单个迭代内交付。

2. 优先级评估与排序

Product Owner 利用 CodeArts Req 的优先级字段，对用户故事进行双维度评估：**业务价值**：节能、碳排可视化、生产协同贡献；**实现复杂度**：技术难度、依赖链、资源消耗。

基于这两个维度构建优先级矩阵，并在 CodeArts Req 中进行标记（P1 - P4），明确各功能的重要性和实现顺序。

（2）核心功能范围

P1：能耗全局监控（高价值/中复杂度）

提供管理层实时监控：瞬时能耗、累积能耗、趋势分析、碳排估算。

P2：异常告警与解释(NLG) (高价值/中复杂度)

自动识别能耗异常（如突增或预测超标），并生成自然语言解释。

P3：生产调度优化(中高价值/高复杂度)

基于历史数据和 LSTM 模型进行能耗预测，为生产计划调整与节能策略提供

建议。依赖稳定的数据采集和计算模块。

P4：运维监控与系统管理(中价值/中复杂度)

包括设备运行状态监控、数据质量校验、系统配置与权限管理等后台功能。

(3) 任务拆分与迭代规划（使用CodeArts Pipeline + 燃尽图）

1. 任务拆分

1) 在 CodeArts Req 中,将每个用户故事进一步拆解为具体任务 (Task) 或技术子任务 (Sub-task)。

2) 每个任务在 **CodeArts Repo** 中关联具体代码分支 (Branch), 形成开发可追踪链路。

3) 为任务分配负责人 (Assignee)、估算 Story Point (或工时)、设置截止时间。

2. 迭代节奏与目标 (Sprint)

1) Sprint 1:

- 在 CodeArts Req 中细化所有用户故事;
- 在 CodeArts Pipeline 中搭建时序数据库环境;
- 完成初步数据采集原型、能耗展示界面与基础异常检测逻辑。

2) Sprint 2:

- 构建完整业务链路: 采集 → 计算 → 预测 → 异常告警 (含 NLG) → 可视化。
- 使用 CodeArts Pipeline 自动化构建、测试和部署。

3) Sprint 3:

- 完成模块联调与系统测试;
- 现场设备调试;
- 编写用户操作手册、技术文档并生成对外交付资料;
- 在 CodeArts 平台提交验收报告并完成项目复盘。

3. 燃尽图与进度监控

1) 利用 CodeArts 的任务管理面板 (看板 / Backlog 视图) 追踪每个 Sprint 中任务的状态 (To Do → In Progress → Done)。

2) 在每个 Sprint 周期内,通过 CodeArts 的燃尽图 (Burndown Chart) 功能监控剩余工作量 (剩余 Story Point 或任务数)。

3) 每日站会 (Daily Scrum) 更新任务完成情况,并将燃尽图数据同步在 CodeArts Req 看板中,以便全团队实时查看进度和风险。

3.3.3 三次迭代目标:

结合项目范围、用户故事优先级以及核心活动依赖关系,团队制定了覆盖 6 天开发周期的三个迭代总体目标与交付物规划。三个迭代构成了从 需求 → 架构 → 数据采集 → 能耗计算 → 预测 → 告警 → 可视化 → 联调测试 →

验收 的完整交付链路。

Sprint 1 —— 需求细化、系统架构设计与基础设施搭建

总体目标：建立项目的基础能力，包括需求澄清、系统架构冻结、数据库基础环境搭建，为后续功能开发奠定基础。

主要成果与活动：完成需求细化与用户故事编写，明确数据采集范围、约束条件、能耗计算需求，产出用户故事清单。输出系统架构设计，完成数据采集层、计算层、预测算法层、应用展示层的整体架构图。搭建数据库基础环境，设计并部署时序数据库表结构，包括设备 ID、采集时间、能耗值等字段。

迭代交付物：《用户故事清单》、《系统架构设计图》、《数据库部署报告》

Sprint 2 —— 采集接口、能耗计算、预测模型、告警模块与可视化开发

总体目标：完成系统的核心业务功能开发，包括实时采集、能耗计算、预测模型、异常告警和可视化展示，形成可以运行的系统主干流程。

主要成果与活动：数据采集接口开发，实现对温度、压力、功率等原始数据的采集与预处理，是所有后续模块的前置条件。能耗计算模块实现电能、热能、气能耗与综合能耗、碳排放量等计算逻辑。预测模型训练基于历史数据训练混合预测模型，提供未来能耗趋势预测能力。异常能耗检测支持能耗突增、预测超标等规则设定与告警触发。

实时能耗看板实现瞬时能耗数字看板、累积曲线以及设备运行状态展示。告警模块开发支持告警弹窗、短信通知、阈值配置等功能。

迭代交付物：一个可运行的端到端系统主干：“采集 → 计算 → 预测 → 告警 → 展示”

初版前端页面、后台接口与告警通知流程全部可运行

Sprint 3 —— 系统集成、测试验收与文档交付

总体目标：完成系统联调、功能测试、现场设备调试、验收流程与文档交付，实现最终可验收的版本。

主要成果与活动：模块接口联调，检查采集接口、计算模块、预测模块、告警与看板间的数据流是否正确功能测试，执行测试用例，对关键功能进行覆盖性验证。

设备对接调试，连接现场设备，确保采集链路稳定。操作手册编写，包含登录、数据查看、告警处理、看板使用等操作说明。最终验收，与客户完成正式验

收确认。项目文档整理，输出用户手册、维护手册和迭代总结报告。项目复盘会议，总结风险、进度经验，为后续迭代提供参考。

迭代交付物：通过验收的系统、《验收确认单》、《用户手册》、《维护文档》、《最终迭代总结报告》

3.3.4 责任分配与团队协作机制

根据团队成员的技术专长与任务匹配度，团队制定了责任分配方案，该方案遵循“技能匹配、避免单点依赖、全过程透明”的原则，与项目管理目标保持一致，分配如下：



4. 项目计划

4.1 综合管理（可选）

4.1.1 制定项目管理计划——整个项目的计划，包括执行/监控/收尾的做法

为确保“智驭能效 (SmartEnergyMaster) ——大型钢铁厂自适应节能系统”项目能够高效、有序地推进，并实现敏捷开发的核心价值，特制定本项目管理计划。该计划旨在整合项目执行、监控与收尾阶段的核心做法，为团队提供清晰、可操作的行动指南。

(1) 项目方法论：采用 Scrum 敏捷开发框架

鉴于本项目功能模块清晰、技术栈明确且需快速迭代验证，团队决定采用 Scrum 敏捷开发框架。此方法强调短周期交付、持续反馈和灵活应对变化，非常适合本项目的开发模式。

迭代周期 (Sprint)：每个迭代周期设定为 2 天，共规划 3 个迭代

核心角色分工：

产品负责人 (PO)	段冠屹	负责定义产品待办事项列表 (Product Backlog)、确定用户故事优先级、接收并验收团队交付的成果
------------	-----	---

Scrum Master (SM)	徐一	负责组织每日站会、移除团队障碍、确保 Scrum 流程顺利执行，并维护项目管理工具
开发团队	全体成员	作为跨职能团队，共同承担开发、测试与集成的责任

(2) 执行阶段做法

核心原则：“那些将要做这项工作的人应该帮助计划这项工作”。在每次 Sprint 规划会议上，全体团队成员共同参与任务拆解和估算。

领导与协作：项目经理（由 Scrum Master 兼任）将通过以身作则来体现对高质量计划和代码规范的重视。团队将使用统一的 Git 版本管理规范 and 代码提交格式，确保协作顺畅。

沟通机制：每日举行不超过 15 分钟的“每日站会”（Daily Stand-up），每位成员汇报昨日进展、今日计划及遇到的阻碍。所有任务均在 Trello/ClickUp 等项目管理工具中实时追踪。

(3) 监控阶段做法

绩效跟踪：基于已批准的项目管理计划和范围基线（WBS 和 范围说明书）作为衡量标准，通过项目管理工具中的看板实时监控各任务的状态（待处理、进行中、待验收、已完成）。

信息收集与报告：定期（每完成一个 Sprint）生成简短的工作绩效报告，内容包括：已完成的任务、未完成的任务、发现的主要问题、当前进度与计划的偏差分析。

变更控制：所有对项目范围、进度或成本的变更请求，必须通过正式的变更流程。首先由提出者在项目管理工具中创建变更卡，然后在下一次团队会议上进行评估，最终由产品负责人和 Scrum Master 共同审批。未经批准的变更不得实施，以防止“范围蔓延”。

(4) 收尾阶段做法

目标：完成所有预定的开发与测试活动，将可运行的系统原型移交给课程指导老师（作为项目发起人代表）。

主要产出：最终产品移交：向指导老师演示完整的系统功能，并提交可部署的系统源码包和 Docker 镜像。

最终报告：提交包含本次开发启动与计划报告、详细设计文档、测试报告在内的完整项目文档集。

项目文档更新：确保所有项目文档（如 WBS 字典、API 文档、测试用例）均为最终版本，并归档至 GitHub 仓库。

经验总结：召开项目回顾会议，总结本次项目在技术、流程和团队协作方面的经验与教训。

4.2 范围管理

4.2.1 收集需求

本项目的需求收集是一个迭代和渐进的过程，主要采用以下方法

(1) **头脑风暴与研讨会：**通过小组会议，团队成员围绕“项目概述”文件中的功能模块，共同探讨具体实现细节和用户故事，这是最主要的需求来源。

(2) **用户故事分析：**从不同用户角色（管理层、车间操作员、生产调度员、IT运维人员）的角度出发，识别其核心诉求，并将其转化为具体的、可验收的用户故事。

(3) **标杆对照：**参考了现有工业能耗监控系统的通用功能，确保我们的项目具备基础的行业标准能力。

基于以上方法，我们识别出的核心需求如下表所示：

角色	用户故事	验收标准
管理层	作为管理层，我希望能够看到全局的实时能耗和碳排放数据，以便于掌握工厂的整体能源状况并辅助战略决策。	在可视化界面中，能以折线图、饼图等形式展示当日实时功率、当月累计用电量及累计碳排放，数据每 30 秒自动更新，前端同步延迟≤1 分钟。
车间操作员	作为车间操作员，我希望系统能在设备出现异常能耗时及时发出告警，提醒我排查问题，避免能源浪费和设备损坏。	当设备数据偏离预设阈值 10%且持续 5 分钟时，系统弹窗和语音同时触发告警。弹窗内容需包含“异常描述+可能原因+建议措施”三部分。
生产调度员	作为生产调度员，我希望系统能提供能耗与产能的优化决策建议，帮助我在保证生产安全和质量的前提下，实现能耗最优。	系统能根据预测模型输出未来 30 分钟的能耗趋势，并在满足约束条件（炉温区间、产量要求）下，给出调整鼓风机功率和冷却水流量的具体百分比建议。
IT 运维人员	作为 IT 运维人员，我希望能够监控系统的健康状态，确保软件平台稳定运行。	系统提供一个独立的监控页面，能显示后端服务、数据库、消息队列等关键组件的运行状态（在线/离线）和资源占用情况（CPU、内存）。

4.2.2 范围定义——项目范围说明书

本项目范围说明书旨在为项目团队和干系人提供一个关于项目边界和可交付成果的共同理解。

项目名称：智驭能效（SmartEnergyMaster）——大型钢铁厂自适应节能系统

产品范围描述： 本项目旨在开发一套针对大型钢铁厂高炉车间的智能化节能系统。系统通过采集高炉、鼓风机、冷却系统等设备的实时传感器数据，计算综合能耗，并利用深度学习模型进行短期能耗预测。基于预测结果，系统将实现

自适应优化控制，并对异常能耗事件进行智能告警。最终，通过可视化界面为不同角色的用户提供决策支持。

项目可交付成果：

- 1. 可运行的系统原型**
 - 1) 包含数据采集、能耗计算、可视化监控、异常预警等核心功能
 - 2) 利用 CodeArts Pipeline 进行构建与自动化部署，确保原型可重复发布
- 2. 完整的项目源代码（前端 + 后端）**
 - 1) 存放于 CodeArts Repo，包含 Git 分支策略与提交规范
 - 2) 支持 CodeArts 的代码评审（Code Review）与合规扫描
- 3. 完整的项目文档集**
 - 1) 包括《项目报告》《系统设计文档》《测试报告》《API 接口文档》等
 - 2) 通过 CodeArts Wiki 统一维护、版本管理，便于团队协作与共享

验收标准：

所有在 WBS 字典中列出的可交付工作包均已按验收标准完成并通过测试
系统可在 Docker 环境中成功部署并正常运行。
项目文档齐全、内容准确，可供他人复现部署过程

项目边界：

包含：

高炉、鼓风机、冷却系统等关键设备的数据采集与处理
基于物理公式的电能、热能、煤气能耗计算
实时能耗与碳排放的可视化监控
基于阈值的简单异常检测与告警
系统基础架构搭建（Spring Boot + Vue + InfluxDB）

不包含：

复杂的深度学习模型（LSTM/Transformer）的训练与调优（仅做模型集成与模拟预测）
自适应优化控制算法的实际执行与反向控制指令下发（仅做算法研究与模拟，强调“人机协同”）
与钢铁厂真实 PLC/DCS 系统的对接（本次使用模拟数据流）
NLG（自然语言生成）引擎的自主开发（采用模板填充式报告生成）

约束：

项目总周期为 6 天（3 个 Sprint，每个 2 天）
团队成员投入时间为课余时间，每人平均每天约 2-4 小时
技术栈限定为 Java(Spring Boot)、JavaScript(Vue.js)、InfluxDB

项目假设：

团队成员已具备 Java、Vue 和数据库的基本开发能力
所需开发工具（IntelliJ IDEA, Node.js, Docker 等）可顺利安装和使用
模拟数据可以有效替代真实传感器数据用于功能验证

4.2.3 创建WBS——WBS结构树及WBS字典

一、Wbs 结构树

1. 项目启动与准备

- 1.1 制定并签署项目章程
- 1.2 明确团队角色与职责
- 1.3 确立敏捷流程与协作工具

2. Sprint 1: 基础架构与核心监控

- 2.1 系统环境与架构搭建
 - 2.1.1 开发环境准备
 - 2.1.2 版本管理与协作规范
 - 2.1.3 项目管理工具配置
 - 2.1.4 Spring Boot 后端框架搭建
 - 2.1.5 Vue 前端项目初始化
 - 2.1.6 数据库表结构设计与 ER 建模
- 2.2 核心功能开发
 - 2.2.1 数据采集与处理模块开发
 - 2.2.2 管理层全局能耗监控模块开发
 - 2.2.3 车间操作员实时异常预警模块开发 (基于静态阈值)
- 2.3 集成、测试与部署
 - 2.3.1 单元测试与接口测试
 - 2.3.2 Docker 环境搭建与系统部署验证
 - 2.3.3 Sprint 1 评审与验收 (里程碑)

3. Sprint 2: 预测与优化模块开发

- 3.1 能耗预测模型集成
 - 3.1.1 准备历史时序数据集并进行特征工程
 - 3.1.2 搭建并训练 LSTM/Transformer 混合预测模型
 - 3.1.3 封装模型为 RESTful API 接口供后端调用
 - 3.1.4 实现未来 30 分钟能耗趋势曲线可视化展示
- 3.2 自适应优化控制算法开发
 - 3.2.1 定义优化目标函数 (最小化总能耗 + 安全/质量惩罚项)
 - 3.2.2 实现强化学习 (RL) 或模糊自适应控制算法逻辑
 - 3.2.3 开发控制建议生成接口 (输出: 风机功率调整%、冷却水流量调整%)
 - 3.2.4 实现人机协同机制 (操作员确认/授权自动执行)
- 3.3 生产调度员决策支持模块开发
 - 3.3.1 开发生产调度员专用可视化界面 (含预测曲线、风险区间着色)
 - 3.3.2 实现“能耗-产能”多维度对比分析图表
 - 3.3.3 集成并展示自适应优化控制模块的决策建议
- 3.4 智能诊断与 NLG 决策报告模块开发
 - 3.4.1 集成并消费能耗预测模块的输出结果 (结合阈值法)
 - 3.4.2 设计并实现 NLG 报告模板 (包含异常描述、原因分析、优化建议)
 - 3.4.3 实现一键生成并导出智能诊断报告 (PDF 格式)
 - 3.4.4 Sprint 2 评审与验收 (里程碑)

4. Sprint 3: 系统集成、测试与收尾

4.1 系统整体集成与联调

4.1.1 集成所有模块（数据采集、预测、优化、告警、可视化）

4.1.2 解决模块间接口兼容性问题

4.1.3 进行端到端（E2E）功能测试

4.2 系统全面测试与缺陷修复

4.2.1 执行系统集成测试

4.2.2 执行压力测试与稳定性测试

4.2.3 修复测试中发现的所有缺陷

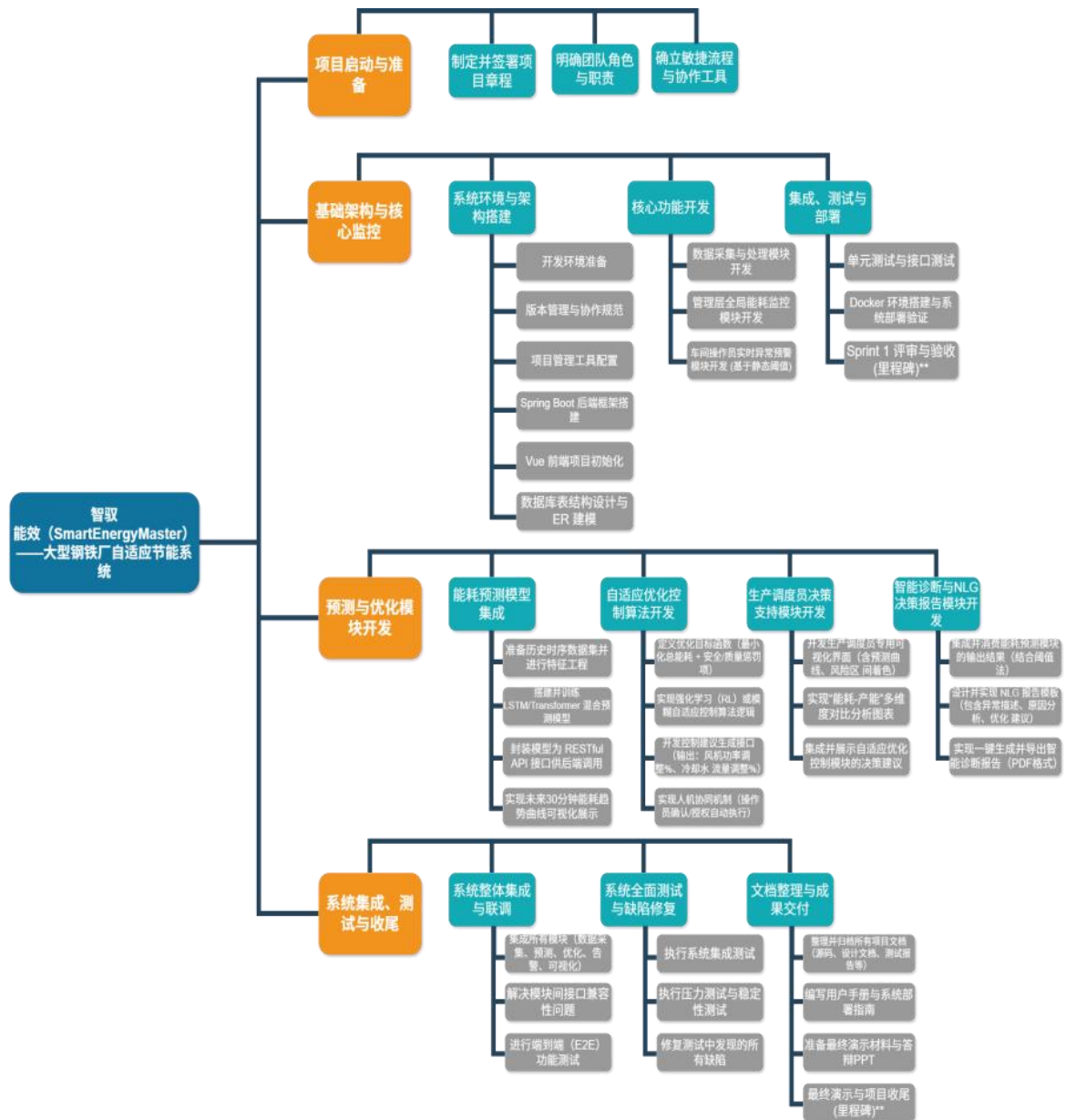
4.3 文档整理与成果交付

4.3.1 整理并归档所有项目文档（源码、设计文档、测试报告等）

4.3.2 编写用户手册与系统部署指南

4.3.3 准备最终演示材料与答辩 PPT

4.3.4 最终演示与项目收尾（里程碑）



二、wbs 字典

WBS 编号	工作内容	负责人	主要输出	验收标准
1.1	制定并签署项目章程	段冠屹 (PO)	《智驭能效项目章程》文档	项目章程文件已创建，明确包含项目目标、范围、关键干系人及对项目经理的授权，并由全体成员在会议中口头确认。
1.2	明确团队角色与职责	段冠屹 (PO) / 徐一 (SM)	团队角色与职责分工表	产品负责人(PO)、Scrum Master(SM)、开发团队成员的角色与核心职责已在会议中明确界定，并记录在案，团队成员均已知晓并认可。
1.3	确立敏捷流程与协作工具	徐一 (SM)	《敏捷开发流程与协作规范》文档	Scrum 框架下的 Sprint 周期（2 天）、每日站会机制、Trello/ClickUp 任务看板使用规则等已确定，并形成书面文档或会议纪要。
2.1.1	开发环境准备	徐一 (SM)	开发环境安装指南、本地环境验证报告	每位成员本地环境能成功运行后端服务（端口正常监听）、前端项目（页面正常渲染），可连接 MySQL 与 InfluxDB 并执行基础 SQL 操作。
2.1.2	版本管理与协作规范	徐一 (SM)	《Git 版本管理与协作规范》文档、GitHub 仓库初始化截图	Git 分支结构清晰（如 feature/, bugfix/），成员能按照规范创建分支、提交代码（信息格式正确）、拉取与合并分支，无代码冲突风险。
2.1.3	项目管理工具配置	徐一 (SM)	Trello/ClickUp 任务看板截图、任务状态说明文档	任务看板包含首个迭代所有任务，任务已分配至对应成员，成员能查看任务详情、更新任务状态，工具通知功能正常。
2.1.4	Spring Boot 后端框架搭建	朱靖轩、万洋	后端项目源代码、框架启动日志截图	后端项目能通过 Maven 构建成功，启动后无报错，基础控制器能返回"Hello World"等测试数据。
2.1.5	Vue 前端项目初始化	朱靖轩、万洋	前端项目源代码、项目结构说明文档	前端项目能通过 npm run serve 启动，页面正常渲染，路由跳转功能正常，ECharts 图表组件能正常展示测试数据。
2.1.6	数据库表结构设计与 ER 建模	朱靖轩、万洋	数据库表结构 SQL 脚本、ER 图（可视化文件）	SQL 脚本能成功执行创建设备表、能耗数据表、告警表、用户表，字段类型与约束设置合理，ER 图能清晰展示表间关系。

2.2.1	数据采集与处理模块开发	李文卓、徐亿立	数据采集接口代码、数据预处理脚本、数据入库日志	采集接口能稳定接收模拟传感器数据（每 30 秒一次），预处理后数据无异常值，可在 InfluxDB 中查询到实时与历史数据，数据存储延迟 ≤ 10 秒。
2.2.2	管理层全局能耗监控模块开发	李文卓、徐亿立	前端界面代码、后端统计接口代码、PDF 报告模板	界面数据每 30 秒自动更新，前端同步延迟 ≤ 1 分钟；点击“导出报告”可生成包含当日实时功率、当月累计用电量及累计碳排放的 PDF 文件，报告内容准确无误。
2.2.3	车间操作员实时异常能耗预警模块开发 (基于静态阈值)	彭锐豪	异常检测算法代码、告警界面代码、语音播报脚本、告警日志表数据	当设备数据偏离预设阈值 10%且持续 5 分钟时，系统弹窗和语音同时触发告警。弹窗内容需包含“异常描述+可能原因+建议措施”三部分。操作员点击“确认处理”后，告警状态更新为“处理中”，日志记录包含时间、用户与操作信息。
2.3.1	单元测试与接口测试	王鑫瑶	测试用例集（含执行结果）、测试报告	测试用例覆盖首个迭代所有核心功能，通过率 $\geq 95\%$ ，无严重 Bug（如系统崩溃、数据丢失）。
2.3.2	Docker 环境搭建与系统部署验证	王鑫瑶	Dockerfile、部署文档、部署验证录屏	Docker 环境中系统可正常登录，所有功能模块能正常运行，部署文档清晰详细，可指导他人重复部署。
2.3.3	Sprint 1 评审与验收 (里程碑)	段冠屹 (PO)	Sprint 1 功能演示报告、干系人反馈记录	产品负责人（代表课程指导老师）对 Sprint 1 完成的功能表示满意，并在验收单上签字确认，同意进入下一个迭代。
3.1.1	准备历史时序数据集并进行特征工程	万洋	清洗后的 CSV/JSON 数据集、特征列表文档	获得过去 60 天的高炉运行数据，完成缺失值填充、异常值剔除，并提取出温度、压力、负载、风机功率等关键特征。
3.1.2	搭建并训练 LSTM/Transformer 混合预测模型	万洋	训练好的模型文件 (.h5/.pkl)、模型性能评估报告	模型在验证集上的均方根误差(RMSE)低于预设阈值，能够输出未来 30 分钟的能耗预测曲线。
3.1.3	封装模型为 RESTful API 接口供后端调用	朱靖轩	模型 API 接口代码、API 文档	后端服务可通过 HTTP POST 请求向/api/predict 发送过去 60 分钟的数据，获得未来 30 分钟的预测结果，响应格式为 JSON。

3.1.4	实现未来 30 分钟能耗趋势曲线可视化展示	徐亿立	前端预测曲线组件代码	在生产调度员界面上，能以折线图形式展示未来 30 分钟的预测能耗曲线，并对超出安全区间的部分进行红色着色标记。
3.2.1	定义优化目标函数(最小化总能耗 + 安全/质量惩罚项)	万洋	优化目标函数数学公式、参数说明文档	目标函数 $\text{Minimize } E_{\text{total}} + \lambda_1 * \text{Penalty}_{\text{quality}} + \lambda_2 * \text{Penalty}_{\text{safety}}$ 已明确定义，其中 λ_1, λ_2 为权重系数。
3.2.2	实现强化学习 (RL)或模糊自适应控制算法逻辑	朱靖轩	控制算法核心代码、算法流程图	算法能根据当前状态和预测结果，计算出使目标函数最优的鼓风机功率和冷却水流量调整方案。
3.2.3	开发控制建议生成接口(输出: 风机功率调整%、冷却水流量调整%)	朱靖轩	控制建议 API 接口代码	后端提供 <code>/api/recommendation</code> 接口，返回类似 <code>{"fan_power_change": -3%, "cooling_flow_change": +2%}</code> 的 JSON 数据。
3.2.4	实现人机协同机制(操作员确认/授权自动执行)	彭锐豪	人机协同界面代码、权限管理逻辑	操作员收到控制建议后，可以选择“接受并手动执行”、“接受并授权自动执行”或“拒绝”。选择授权后，系统将指令下发至 PLC 模拟器。
3.3.1	开发生产调度员专用可视化界面(含预测曲线、风险区间着色)	徐亿立	生产调度员界面源代码	界面整合了实时能耗、预测曲线、产能数据，布局清晰，风险区间有明显视觉提示。
3.3.2	实现“能耗-产能”多维度对比分析图表	徐亿立	多维分析图表组件代码	提供柱状图或热力图，可对比不同班次、不同生产线的能耗与产能比值，帮助发现效率瓶颈。
3.3.3	集成并展示自适应优化控制模块的决策建议	徐亿立	决策建议集成代码	在生产调度员界面显著位置，动态显示来自 <code>/api/recommendation</code> 接口的控制建议。
3.4.1	集成并消费能耗预测模块的输出结果(结合阈值法)	彭锐豪	增强型异常检测算法代码	新的异常检测不仅检查绝对阈值，还检查实际数据与预测数据的残差是否过大，提高了告警的准确性。
3.4.2	设计并实现 NLG 报告模板(包含异常描述、原因分析、优化建议)	彭锐豪	NLG 报告模板库、数据填充逻辑	模板包含“设备 X 在 Y 时间出现 Z 问题，主要原因可能是 A，建议采取 B 措施”的结构，能根据具体数据填充变量。
3.4.3	实现一键生成并导出智能诊断报告(PDF 格式)	彭锐豪	PDF 报告生成接口代码	点击“生成诊断报告”按钮，系统能调用 NLG 引擎生成报告文本，并将其导

				出为带图表的 PDF 文件。
3.4.4	Sprint 2 评审与验收 (里程碑)	段冠屹 (PO)	Sprint 2 功能演示报告、干系人反馈记录	产品负责人（代表课程指导老师）对 Sprint 2 完成的预测、优化和智能诊断功能表示满意，并在验收单上签字确认，同意进入收尾阶段。
4.1.1	集成所有模块（数据采集、预测、优化、告警、可视化）	全体成员	集成后的完整系统源码	所有模块在同一个 Docker 容器或开发环境中能够协同工作，数据流畅通无阻。
4.1.2	解决模块间接口兼容性问题	朱靖轩、万洋	接口联调记录、修复后的代码	消除了因数据格式不匹配、API 版本不一致等原因导致的集成错误。
4.1.3	进行端到端（E2E）功能测试	王鑫瑶	E2E 测试用例、测试录屏	模拟一个完整的业务场景（如：数据采集->预测->生成建议->操作员确认->生成报告），整个流程能顺利完成。
4.2.1	执行系统集成测试	王鑫瑶	集成测试报告	对各模块组合后的功能进行了全面测试，未发现新的集成层面的缺陷。
4.2.2	执行压力测试与稳定性测试	王鑫瑶	压力测试报告	系统在连续运行 24 小时或承受高并发数据写入的情况下，CPU 和内存占用平稳，无崩溃或严重延迟。
4.2.3	修复测试中发现的所有缺陷	全体成员	Bug 修复清单、更新后的代码	根据测试报告，所有已知缺陷（Bug）均已修复，系统达到稳定可用状态。
4.3.1	整理并归档所有项目文档（源码、设计文档、测试报告等）	王鑫瑶	项目文档归档包	所有文档按“需求、设计、开发、测试、计划”分类整理，上传至 GitHub 仓库的 docs 目录下。
4.3.2	编写用户手册与系统部署指南	王鑫瑶	《用户操作手册》、《系统部署指南》	用户手册详细说明了各功能的使用方法；部署指南清晰列出了从零开始搭建和运行本系统的步骤。
4.3.3	准备最终演示材料与答辩 PPT	段冠屹 (PO)	最终演示 PPT、演示脚本	PPT 内容涵盖项目背景、目标、技术架构、核心功能演示、创新点和总结，可用于课堂答辩。
4.3.4	最终演示与项目收尾 (里程碑)	全体成员	最终系统、完整项目文档集、结题报告	课程指导老师对最终交付成果进行评分，项目正式关闭。

4.3 进度管理

4.3.1 活动定义

基于本项目的 Scrum 敏捷开发流程（3 个迭代，每个迭代 2 天，总工期 6 天），拆解 16 项核心活动，覆盖项目组织与开发全流程（含需求、设计、开发、测试、交付），确保与 WBS 层级匹配且非仅软件功能模块，具体如下：

活动 ID	活动名称	活动描述	所属迭代	对应 WBS 层级
A1	需求细化与用户故事编写	确认数据采集范围、优化约束条件	迭代 1	2.1
A2	系统架构设计	设计数据采集层、计算层、算法层、应用层架构	迭代 1	2.2
A3	时序数据库搭建	设计数据表结构（含设备 ID，采集时间，能耗值）	迭代 1	2.3
A4	数据采集接口开发	开发数据对接接口，实现温度、压力等数据实时采集与预处理	迭代 2	3.1
A5	能耗计算模块开发	实现公式实现电 / 热 / 气能耗及综合能耗、碳排放量计算	迭代 2	3.2
A6	能耗预测模型训练	基于历史数据训练混合预测模型，优化预测精度	迭代 2	3.3
A7	异常能耗检测开发	设置能耗突增，预测超标等告警规则	迭代 2	3.4
A8	实时能耗看板开发	开发瞬时能耗数字展示，累积能耗曲线，设备状态图	迭代 2	3.5
A9	告警模块开发	实现告警弹窗、短信通知功能	迭代 2	3.6
A10	模块接口联调	验证各模块数据流转正确性，修复接口格式、数据延迟	迭代 3	4.1
A11	功能测试用例执行	执行核心用例进行集中测试	迭代 3	4.2
A12	设备对接调试	连接设备，验证数据采集稳定性	迭代 3	4.3
A13	编写操作手册	演示登录，数据查看，告警处理等操作	迭代 3	4.4
A14	验收确认	验收核心功能	迭代 3	4.5
A15	项目文档整理	编写用户手册、维护文档、迭代总结报告	迭代 3	5.1

4.3.2 活动排序

基于三个迭代的紧凑关系，采用前导图法（PDM）排序，确保并行高效，串行→→衔接：

1. 迭代内活动依赖

- (1) 迭代 1：A1（需求）→ A2（架构），A3（数据库）与 A2 并行（A2 上午输出架构，A3 下午同步部署数据库），3 项活动 1.5 天完成，剩余 0.5 天用于迭代 1 复盘。
- (2) 迭代 2：A4（采集接口）为核心前置活动（需先获取数据），A5（能耗计算）、A6（预测模型）、A7（告警规则）、A8（看板开发）、A9（通知模块）与 A4 并行 6 项活动 2 天内通过 3 人并行实现。
- (3) 迭代 3：A10（接口联调）→ A11（功能测试）→ A12（现场调试）→ A13（操作手册）→ A14（验收），A15（文档）、A16（复盘）与 A14 并行（A14 上午验收，A15 下午整理文档、A16 下午开会），7 项活动 2 天内串行 + 并行完成。

2. 迭代中间依赖

迭代 1（A1-A3）全部完成 → 迭代 2（A4-A9）启动（需架构、数据库就绪）；
迭代 2（A4-A9）全部完成 → 迭代 3（A10-A16）启动（需核心模块开发完成）

3. 关键里程碑

- (1) M1：迭代 1 交付（第 2 天），输出《用户故事清单》《架构设计图》《数据库部署报告》
- (2) M2：迭代 2 交付（第 4 天），输出可运行的“采集 + 计算 + 预测 + 告警 + 展示”核心模块
- (3) M3：项目交付（第 6 天），输出通过验收的系统 + 《验收确认单》《用户手册》

4.3.3 活动资源估算

采用自下而上估算方法，结合项目需求与团队能力，确定各活动所需资源（人力、硬件、工具，如[表 4.3.2](#)

活动 ID	人力资源(角色/人数)	硬件资源	工具/技术资源	迭代内资源分配
A1	产品负责人 1 名	办公电脑	TAPD（用户故事管理） 腾讯会议	第 1 天上午（2 小时）
A2	架构师 1 名	办公电脑	DrawIO（架构图）	第 1 天上午 - 下午（4 小时）
A3	运维工程师 1 名	服务器	Docker InfluxDB 官方文档	第 1 天下午 - 第 2 天上午（6 小时）
A4	后端开发工程师 1 名	办公电脑 PLC 模拟	Python Postman	第 3 天全天（8 小时）

		器		
A5	后端开发工程师 1 名	办公电脑	Python 单元测试框架 pytes	第 3 天下午 - 第 4 天上午 (6 小时)
A6	算法工程师 1 名	GPU 服务器	TensorFlow Lite Jupyter Notebook	第 3 天下午 - 第 4 天上午 (6 小时)
A7	算法工程师 1 名	测试电脑	Excel (告警规则表) 接口测试工具	第 4 天上午 (2 小时)
A8	前端开发工程师 1 名	开发电脑	Vue3 ECharts	第 4 天全天 (8 小时)
A9	后端开发工程师 1 名	开发电脑	短信 API 对接文档 Python requests 库	第 4 天下午 (3 小时)
A10	前端开发工程师 1 名 后端开发工程师 1 名	开发电脑 测试服务器	日志分析工具 接口调试工具	第 5 天上午 (4 小时)
A11	测试工程师 1 名	测试电脑 测试服务器	JMeter 测试用例管理工具	第 5 天上午 - 下午 (6 小时)
A12	运维工程师 1 名 开发工程师 1 名	笔记本电脑 现场设备	Xshell Modbus 调试工具	第 5 天下午 - 第 6 天上午 (6 小时)
A13	产品负责人 1 名	笔记本电脑 投影设备	培训 PPT 系统演示环境	第 6 天上午 (2 小时)
A14	产品负责人 1 名 需求人员 1 名	笔记本电脑 GPU	核心功能演示脚本	第 6 天上午 (2 小时)
A15	文档工程师 1 名	办公电脑	Markdown Excel	第 6 天下午 (3 小时)
A16	全团队 (8 人)	办公电脑	腾讯会议复盘	第 6 天下午 (1 小时)

表格 4.3.2

4.3.4 活动工期估计

采用三时估计法 (乐观时间 a、最可能时间 m、悲观时间 b) 计算期望工期 $(a+4m+b)/6$, 工期单位为“小时”, 具体如[表 4.3.3:](#)

表格 4.3.3

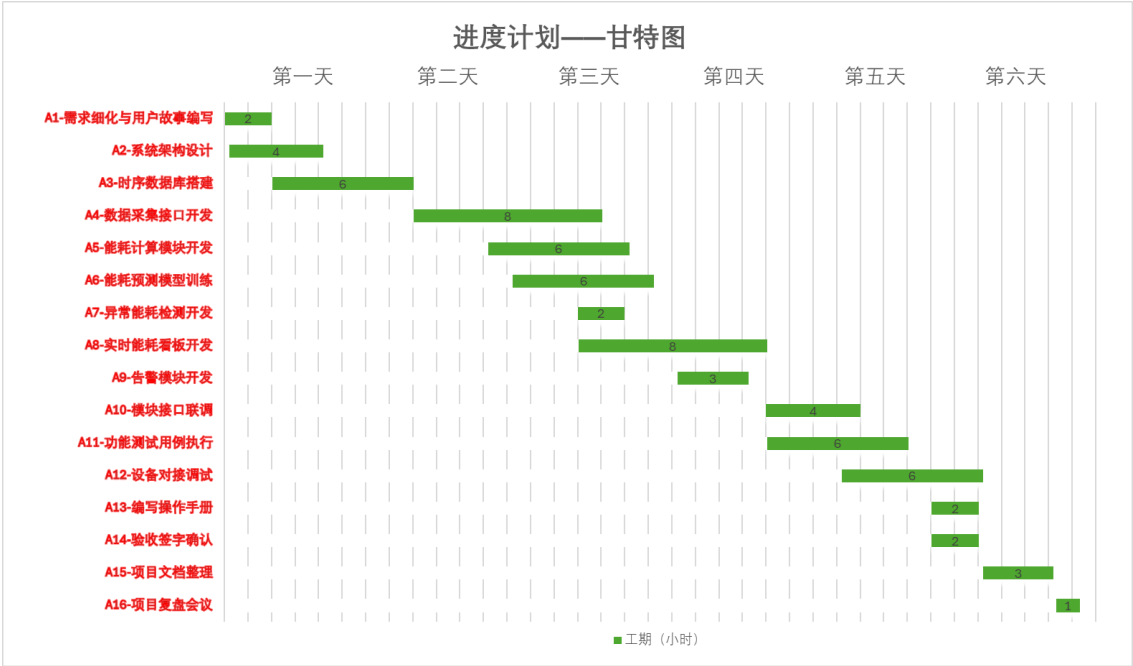
活动 ID	乐观时间 (a)	最可能时间 (m)	悲观时间 (b)	期望工期	所属迭代	工期优化说明
A1	1	2	3	2	迭代 1	聚焦核心需求, 避免冗余讨论
A2	3	4	5	4	迭代 1	复用工业软件架构模板
A3	4	6	8	6	迭代 1	Docker 一键部署, 减少配

						置时间
A4	6	8	10	8	迭代 2	基于开源 Modbus 库开发
A5	4	6	8	6	迭代 2	能耗公式模块化编码，减少重复工作
A6	4	6	8	6	迭代 2	采用轻量化模型，训练时间缩短
A7	1	2	3	2	迭代 2	参考行业告警阈值，快速配置
A8	6	8	10	8	迭代 2	使用现成 ECharts 组件，快速搭建
A9	2	3	4	3	迭代 2	调用成熟短信 API，减少开发量
A10	3	4	5	4	迭代 3	优先联调核心链路，次要接口后续迭代
A11	4	6	8	6	迭代 3	核心用例优先执行，次要用例抽样
A12	4	6	8	6	迭代 3	提前确认现场 IP、端口，减少调试时间
A13	1	2	3	2	迭代 3	优化手册
A14	1	2	3	2	迭代 3	提前准备验收脚本
A15	2	3	4	3	迭代 3	用标准化文档模板，快速填充内容
A16	0.5	1	1.5	1	迭代 3	聚焦关键问题，避免冗长讨论

迭代周期汇总

1. 迭代 1（12 小时）：A1（2h）+ A2（4h）+ A3（6h），无冗余时间
2. 迭代 2（40 小时）：A4（8h）+ A5（6h）+ A6（6h）+ A7（2h）+ A8（8h）+ A9（3h），2 天完成
3. 迭代 3（30 小时）：A10（4h）+ A11（6h）+ A12（6h）+ A13（2h）+ A14（2h）+ A15（3h）+ A16（1h），2 天完成

4.3.5 进度计划——甘特图



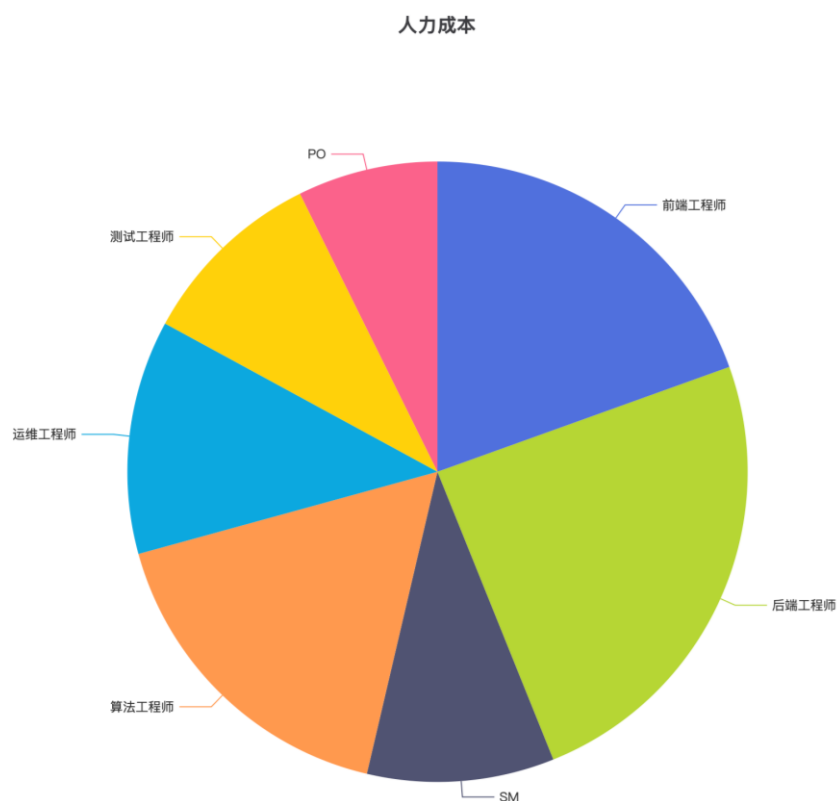
4.4 成本管理

4.4.1 成本预估

(1) 人力成本（时间成本）

角色	主要任务	迭代投入（小时）	等效工时
PO	需求澄清、用户故事、验收	6h	每迭代参与 2h
SM	会议组织、风险管理、任务跟踪	8h	每迭代每日站会+协调
后端工程师	采集接口、能耗计算、告警接口	20h	A4/A5/A9/部分联调
前端工程师	看板、告警 UI、可视化	16h	A8/A10
算法工程师	LSTM 模型、异常检测规则	14h	A6/A7
运维工程师	InfluxDB、Docker 部署，联调支持	10h	A3/A12
测试工程师	测试用例、接口测试	8h	A11

总人力投入：约 82 小时（≈ 10.25 人日）



（2）软件资源成本

工具/环境	成本	备注
Python / TensorFlow Lite	0 元	开源
Node.js / Vue3 / ECharts	0 元	开源
Docker / Docker Compose	0 元	开源
InfluxDB（开源版）	0 元	免费
GitHub	0 元	Free Plan
VSCode / IDEA（社区版）	0 元	免费

软件资源成本估算：0 元

（3）硬件资源成本

资源	成本	说明
本地开发环境	0 元	学生自有设备
GPU 服务器	100 元	租用服务器来进行LSTM训练
数据存储（InfluxDB）	0 元	本地+云端存储量足够

硬件成本估算：100 元

(4) 杂项与数据获取成本

项目	金额（元）	说明
云盘同步/备份	20 元	如使用 OneDrive/百度网盘
打印、资料	20 元	验收材料、图表打印
杂项	30 元	会议记录、附件整理

杂项成本估算：70 元

成本估算汇总

项目	成本	备注
人力成本	82 小时	课堂项目
软件成本	0 元	使用开源工具
硬件成本	100 元	租用服务器
杂项	70 元	数据与材料

最终成本估算：约 170 元（实际费用） + 82 小时（等效人力成本）

4.4.2 成本预算

(1) 预算分配表

成本项	预算	控制策略
杂项支出	70 元	成员共享云盘、共用打印材料
工时预算	三个迭代/6 天	依据 WBS 严格限制任务范围
算法训练时间	6 小时	使用轻量化模型，减少 GPU 使用
环境部署	100 元	统一 Docker 脚本一次部署、全员复用

(2) 成本控制策略

结合讨论五的风险管理成果（如 Docker 冲突、接口延迟、预测模型失败等），制定如下成本控制措施：

1.统一环境配置：使用统一 Docker 镜像，避免“配置 8 次 → 只跑一次”的浪费。

2.冻结用户故事：Sprint 内禁止增加新需求，防止返工造成工时浪费。

3.自动化测试减少重复劳动：前后端接口统一使用 Swagger 文档，减少联调成本。

4.算法训练采用小样本快速试验：提前验证模型结构，缩短 GPU 占用时间。

5.定期燃尽图检查：SM 每日跟踪工时，过载任务提前拆分。

4.5 质量管理

4.5.1 质量管理计划

质量管理计划围绕以下两个方面展开：过程质量（Process Quality）、产品质量（Product Quality）

过程质量保证措施：

过程环节	质量措施	效果
需求质量	用户故事使用 INVEST 原则；验收标准使用 GWT（Given-When-Then）格式	避免需求不清导致返工
迭代质量	每日站会、迭代评审与回顾；燃尽图监控任务量	发现问题及时调整
代码质量	统一风格（ESLint + Pylint），强制 Code Review	降低 Bug 率与合并冲突
接口质量	Swagger 自动生成 API 文档；联调前冻结接口	减少后端/前端反复沟通
测试	单元测试 + 接口测试 + 功能演示，覆盖核心模块	保证首迭交付质量

结合系统 5 大核心模块，有如下产品质量保证措施：

1) 数据采集模块

- 增加异常值过滤与字段校验
- 采集接口加重试机制
- 建立模拟数据源用于压力测试

2) 能耗计算模块

- 能耗公式模块化（电、气、热分开）
- 引入单元测试校验公式正确性
- 使用历史数据对计算结果进行抽样检验

3) 预测模型模块

- 使用轻量化LSTM，避免过拟合
- 引入 Early-Stopping + 小样本预验证
- 使用滚动窗口对预测误差进行持续监控

4) 异常告警模块

- 阈值与残差双判据，减少误报/漏报
- 告警信息采用“三段式结构”：异常→原因→建议
- 告警日志自动记录，方便后续溯源

5) 可视化模块

- 前端采用异步刷新，防止 UI 卡顿
- 使用ECharts大数据模式，减少渲染开销
- 夜间模式与大屏模式进行适配

4.5.2 过程改进计划

(1) 迭代 1 改进点

问题： 数据采集接口联调耗时较长

改进： 在迭代 2 中提前进行接口 Schema 对齐
建立自动化接口 Mock 服务，前端可提前开发

(2) 迭代 1 改进点

问题：模型训练初期精度不稳定

改进： 在迭代 2 中加入“滚动再训练”机制
引入参数搜索脚本，减少人工调参时间

(3) 团队协作改进

问题：任务分配不均导致部分成员在迭代尾声工作量过大

改进： 采用更细粒度的任务拆分
SM 在每日站会中提前发现瓶颈并调整任务

(4) 质量改进

问题： 单元测试覆盖不够

改进： 迭代 2 起要求新增功能必须附带测试
覆盖关键公式、核心预测 API、告警模块

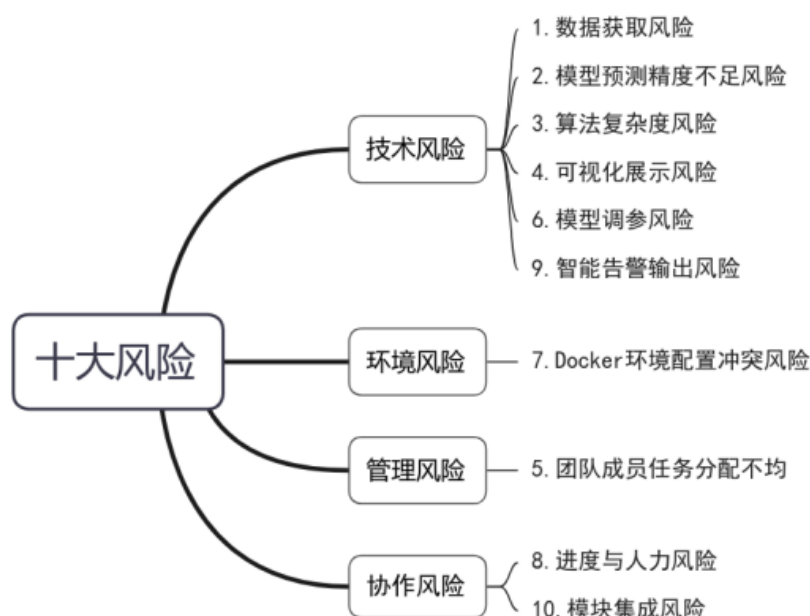
4.6 风险管理

4.6.1 识别风险

团队通过头脑风暴方法识别出技术风险、环境风险、管理风险与协作风险四大类共 20+ 项潜在风险。最终经团队投票筛选出 10 项高优先级风险。

评估维度主要聚焦：

1. **技术风险**：算法、模型泛化能力、系统兼容性、数据质量等不确定性
2. **环境风险**：部署环境、第三方依赖、硬件性能、网络波动等
3. **管理风险**：进度计划、任务分解、资源分配、沟通效率
4. **协作风险**：跨模块接口、多人并行开发中的依赖关系与冲突

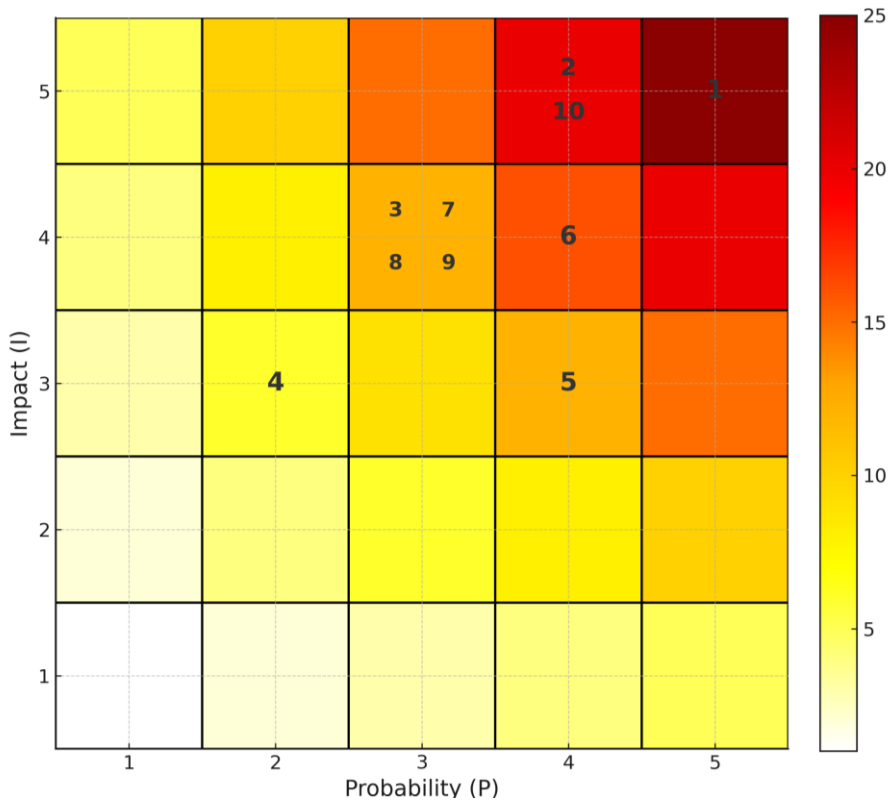


4.6.2 实施定性风险分析

采用概率（P=1~5）与影响（I=1~5）矩阵，对识别的风险进行评估。评估维度包括：概率（P）：低=1，较低=2，中=3，较高=4，高=5；影响（I）：低=1，较低=2，中=3，较高=4，高=5；风险等级 = P × I。筛选规则为风险等级 ≥ 15 的项列为高优先级风险。评估维度主要聚焦：对进度、质量、开发可行性与系统稳定性的影响。

序号	风险项	风险类型	概率	影响	风险等级
1	数据获取风险	技术风险	5	5	25
2	模型预测精度不足	技术风险	4	5	20
3	算法复杂度风险	技术风险	3	4	12
4	可视化展示风险	技术风险	2	3	6
5	团队成员任务分配不均	管理风险	4	3	12
6	模型调参风险	技术风险	4	4	16
7	Docker 环境配置冲突	环境风险	3	5	15
8	进度与人力风险	协作风险	3	4	12
9	智能告警输出风险	技术风险	3	4	12
10	模块集成风险	协作风险	4	5	20

将以上风险等级以风险热力图形式展示如下图所示（以风险编号表示）：



4.6.3 实施定量风险分析

对两类对项目影响最大的风险进行了量化评估：

- (1)数据获取风险:若数据字段缺失或格式不一致,将导致模型预测偏差 $\geq 15\%$,进而使后续功能联调延迟约 0.5~1 天。
- (2)模型训练风险:若模型训练失败比例 $\geq 20\%$,需重新调整参数和训练流程,可能造成整体迭代延期 0.5 天以上。

4.6.4 规划风险应对策略

对每一类风险都制定了风险的应对策略,以及发生相应风险时的责任人

序号	风险名称	应对策略	责任人
1	数据获取不完整或字段不一致	1. 迭代开始前由后端与算法人员共同确认数据字典 2. 建立字段映射脚本,确保输入格式统一 3. 字段缺失时使用插值或合成数据补齐训练集	后端开发 + AI/算法工程师
2	模型预测精度不足	1. 扩充训练样本量,引入多工况历史能耗数据 2. 采用滚动再训练(Rolling Retrain)周期性更新权重 3. 建立预测误差监控,超阈值自动触发微调	AI/算法工程师
3	模型训练与调参效率低、失败风险高	1. 建立自动化超参搜索脚本(Grid Search) 2. 使用 Early Stopping 避免过拟合与训练浪费 3. 小样本快速试验框架验证参数,降低调参成本	AI/算法工程师
4	数据采集接口不稳定	1. 为采集接口加入超时重试与断线重连机制 2. 使用缓存队列(Buffer)避免瞬时丢包 3. 定期进行接口压力测试提前暴露瓶颈	测试/运维 (DevOps) + 后端开发
5	Docker 环境配置冲突	1. 使用统一基础镜像(如 Python + TensorFlow Lite 等固定版本) 2. 维护标准化部署脚本(docker-compose.yml) 3. 锁定团队成员开发环境版本,避免差异导致部署失败	测试/运维 (DevOps)
6	智能告警误报/漏报	1. 构建历史人工校验样例集验证告警阈值 2. 使用动态阈值机制,随生产状态自动调整 3. 完整记录告警日志供后续分析	AI/算法工程师 + 后端开发
7	可视化渲染性能不足	1. 前端采用异步请求 + 局部刷新,减少整页渲染 2. 控制数据推送频率(如每秒 1 次)降低负载 3. 基于 ECharts 大数据模式优化曲线绘制性能	前端开发
8	任务分配不均导致人力瓶颈	1. Scrum Master 每日检查任务燃尽图并动态调度 2. 模块拆解为更细粒度任务,支持多人并行 3. 为关键角色预留缓冲时间应对突发任务	Scrum Master (徐一)
9	模块间接口或数据格式不一致	1. 在迭代 2 前冻结 API 文档与数据字段格式 2. 模块完成后立即执行小规模集成测试 3. 建立统一 JSON Schema 校验机制	后端开发 + 测试/运维
10	迭代进度延期	1. 优先保证关键路径(采集 → 计算 → 预测 → 告警) 2. 设置 10% 时间缓冲区吸收小延期 - 每日站会暴露阻碍,Scrum Master 及时协调资源	Scrum Master (徐一)

团队建立了风险监控机制，包括每日晨会（Daily Scrum）风险通报、每周风险审查（Risk Review），以及关键风险的监控指标（KPI）。当任一指标达到触发条件时，自动进入风险应对流程，由风险责任人负责处理，并在下一次迭代回顾会上进行复盘与记录更新。

4.7 沟通管理

4.7.1 规划项目沟通管理

为确保 SmartEnergyMaster 项目在紧凑的 6 天周期内顺利推进，本项目采用 敏捷 Scrum + 工业软件项目特点结合 的沟通规划，沟通策略重点围绕 需求准确性、模块协同、风险透明度及快速响应设计。

(1) 沟通目标

1. 保证需求一致理解，减少返工
2. 及时暴露技术与进度风险
3. 加快模块间接口协作效率
4. 提升干系人参与度与透明度
5. 确保预测模型、数据采集与告警模块的逻辑对齐

(2) 沟通方式与频率

沟通方式	参与角色	目的与内容	频率	输出物
每日站会	全体成员	同步进展与阻碍	每天 10 分钟	每日站会纪要
接口协调沟通	后端/算法/前端	API 与字段确认	每天 1 - 2 次	接口字段确认表
数据一致性检查	后端/算法	字段与缺失值确认	迭代前	数据字典 V1/V2
需求方同步会议	产品/需求方	进度与原型确认	每两天	阶段进度简报
迭代评审	全体/需求方	展示成果	每迭代结束	评审反馈表
迭代回顾	全体成员	复盘与改进	每迭代结束	回顾建议

(3) 沟通工具规划

工具	适用场景	备注
CodeArts	任务拆解、迭代管理与进度可视化	支持燃尽图、任务透明、多人协作
腾讯会议/微信群	远程实时讨论与问题快速响应	保障跨地域沟通及时
CodeArts Wiki/文档模块	接口文档、设计文档与项目资料共享	文档版本统一，可多人同时编辑
PocessOn	系统架构草图绘制与团队协作	多人实时协作，支持共享

(4) 干系人沟通管理计划

干系人	关注点	沟通内容	沟通模式
需求方（老师）	功能完成度与项目进度	功能演示、能耗预测效果展示、用户故事验收	定期汇报（迭代评审/Sprint Review）
开发团队	任务状态与模块接口	接口变更、任务进度、依赖协调	每日站会（Daily Scrum）
算法工程师	数据质量与模型指标	数据字段映射、异常值处理、预测误差分析	数据专题会议/CodeArts 任务讨论
前端/后端	接口联调与展示效果	页面展示验证、字段匹配、接口调用问题	联调会议/CodeArts Workitem 协作

最终目标是确保所有干系人在整个项目周期内 **信息对称、反馈及时、风险可控**。

4.7.2 启动计划阶段的协作沟通情况

在项目启动与计划阶段（迭代前两天），团队围绕需求、架构、风险与数据进行了高频、务实的沟通，取得了以下成果。

(1) 需求澄清沟通（2 次）

1. 产品负责人组织两次需求讨论会，针对能耗指标、预测范围、告警逻辑等内容进行拆解。

2. 经过讨论后确认系统核心需求为：“采集—计算—预测—告警—展示”五大功能闭环。

输出《用户故事清单 V1》，减少了后续需求歧义。

(2) 技术架构沟通（3 次）

1. 架构师与后端、算法、前端围绕接口格式、数据库结构、模型输入字段进行了三次技术讨论。

2. 确定采用 时序数据库 + Python 接口层 + LSTM 预测模块 + Web 前端可视化 的分层架构方案。

3. 整理出《系统架构图》和《接口字段表 V1》，避免模块间理解不一致。

(3) 风险头脑风暴（1 次）

1. 全体成员使用 Miro 进行风险收集与分类，将 20+ 风险聚类为 4 类（技术/环境/管理/协作）。

2. 经投票确定十大高优先级风险，并生成《风险登记册 v1》。

3. 重点再确认“数据字段缺失风险”和“模型精度不达标风险”，并提前规划应对措施。

(4) 跨模块协作效率提升

1. 后端与算法组提前确认训练数据格式，使迭代 2 模型训练按时开展。
2. 前后端在启动阶段完成能耗看板样式示意图，减少后期沟通成本。

(5) 沟通效果评价

1. 需求理解一致性高（无重大变更）
2. 数据格式在迭代开始前即完成统一，避免返工
3. 风险暴露及时，关键风险均获得对应策略
4. 沟通成本保持在低水平，团队协作节奏稳定

总体而言，启动计划阶段沟通顺畅、节奏紧凑，为后续开发阶段奠定了明确的方向和稳定的协作基础。